



Web Development Technologies

Tutorial and Lab Sheet 1

Introduction to Visual Studio & C# programming

Exercise 1 My first time with Visual Studio 2026

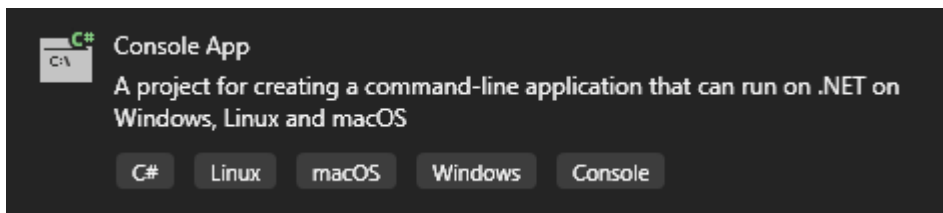
Please download the software required on your laptops. It is **not** available on the lab machines.



Instructions can be found in Canvas → Modules → Pre-reading: IMPORTANT → Software Installation.

Step 1:

Launch Visual Studio 2026 and create a new project using template:



It may differ for mac users: **Web and Console → App → Console Application**

In the **Target Framework** drop-down of the **Configure your new Console Application** dialog, select **.NET 10.0**, and select **Next**.

Call the project as **HelloWorldApplication**.

```
/* My first program in C#. */  
Console.WriteLine("Hello World");  
Console.ReadKey();
```

Run the program by either pressing F5 or selecting **Debug → Start Debugging** menu option. You will now see the output in a console or terminal (*depending upon windows or mac environment*).

Ask the lab assistant for help if you cannot create the project.



Step 2:

Most of the menus that you will see at the top of Visual Studio shall be useful as we traverse through the different stages of this course- *feel free to explore the various menu options*. The ones that will be particularly of interest will be- **File, View, Project, Build, Debug** and **Tools** menu. Pay special attention to these. Now let us look at other components of Visual Studio-

Solution Explorer- Projects (What is that?) and solutions are managed using the Solution Explorer. This is something like the windows explorer and presents the projects and other files in a hierarchical, visual manner. *If it is not visible or for some reason you have accidentally closed it- you can always reset window layout through Tools menu or you can even access it through View → Solution Explorer.*

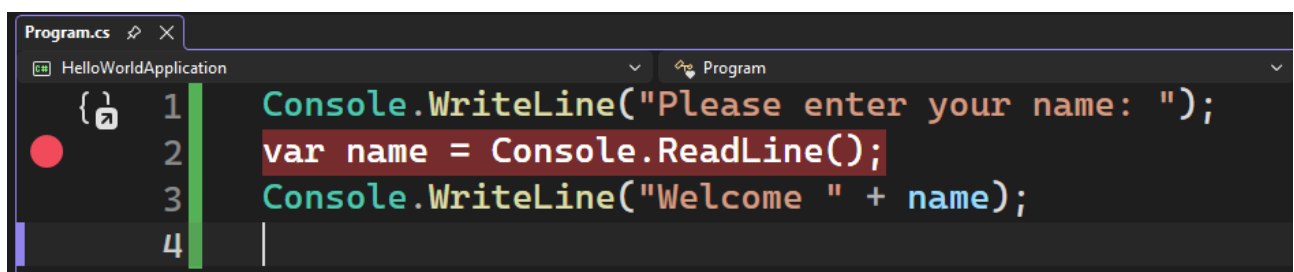
Exercise 2 Debugging in Visual Studio 2026

Now we shall see what debugger does for us:

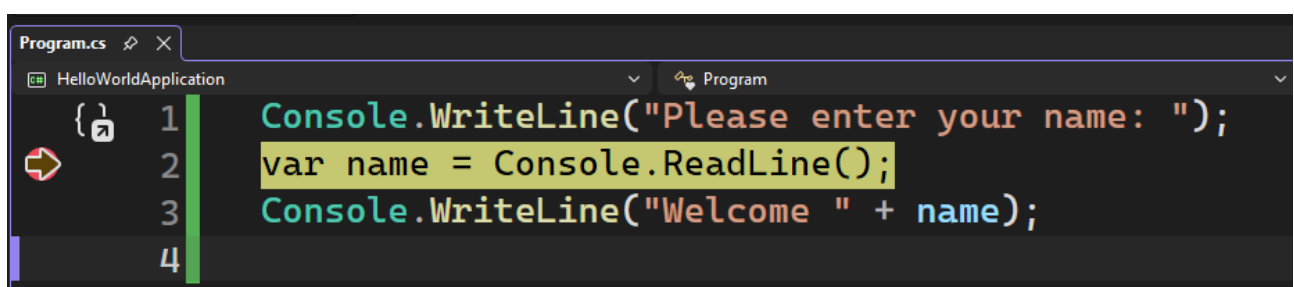
1. Modify the **Main()** method as

```
Console.WriteLine("Please enter your name: ");
var name = Console.ReadLine();
Console.WriteLine("Welcome " + name);
```

2. Run the program with debugging (**Debug → Start** or **F5**). Observe what happens.
3. Find the **ReadLine** method.
4. Click in the margin on left hand side (or use **F9**) to add a break point to the line indicated in the following diagram.



5. Now run the program as (**Debug → Start** or **F5**).
6. Visual Studio will be brought to the front and the code the breakpoint was set on will be highlighted in yellow.



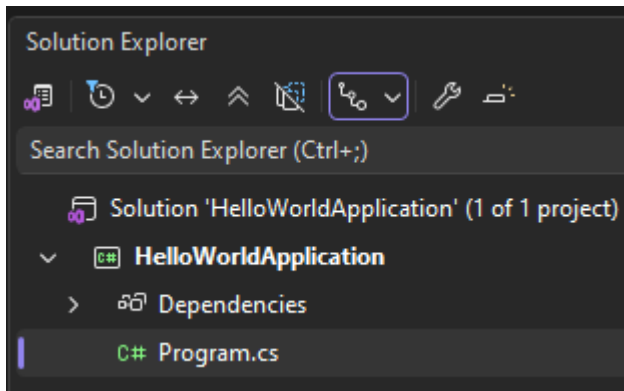
7. Use **F10** (or the functions available from the **Debug** menu) to step through the program. *Note that the first time you press F10 you will take back to the console.*

Exercise 3 Files generated by Visual Studio

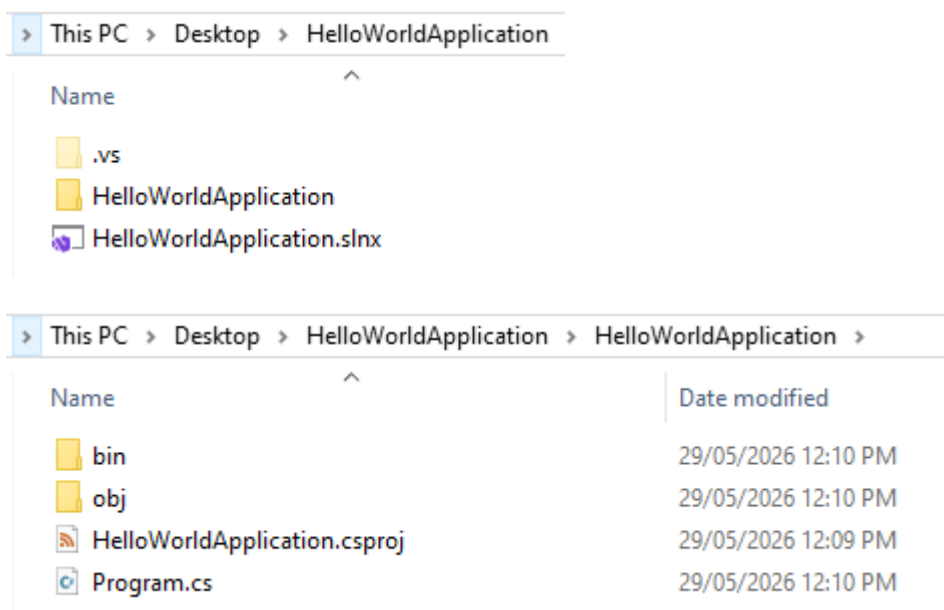
Have a look at the **Solution Explorer** now and explore each file.

Also open the folder where you have stored this console application through File Explorer (Windows) and Finder (Mac). What you will see is:

Project as seen inside Solution Explorer:



Project as seen inside the directory where it is saved:



Note: Your project name and folder may differ in the name.

This may look different on Mac laptops!

Double-clicking on the **.slnx / .sln** solution file will load the project. If time permits, open the **.slnx / .sln** and **.csproj** files in Notepad (or any text editor).

Exercise 4 Object Orientation in C#

- (4.1) Declare a class **Building** which has three *public* data members namely – *floors*, *area* and *occupants*. Then declare a separate class that declares two objects (say *house* and *office*) of the type **Building** and write a driver method to print something like this:

House has:

2 floors
4 occupants
2500 total area
625 area per person

Office has:

3 floors
25 occupants
4200 total area
168 area per person

- (4.2) Now make the data member(s) *private* and add the *properties* to retrieve them.
(4.3) Did you add a constructor to the class declaration? If not, add a default and add a parameterised constructor (*don't fear the big word! -it simply means a constructor with parameters, unlike a default constructor*).

Write appropriate code and test it using Visual Studio.

Exercise 5 Structs vs Classes in C#

What are *structs*? How do they differ from a class? Substantiate your answer with help of an example.

Your tutor will help you.

Exercise 6 Using GitHub with Visual Studio 2026

Pre-requisite: Join the rmit-wdt-s2-2026 Organisation

This document assumes you have created a GitHub account and have accepted an invitation to join the **rmit-wdt-s2-2026** organisation.

If you have not received this email, please contact Matthew at matthew.bolger2@rmit.edu.au

NOTE: Using personal git accounts and not joining above organisation will FETCH a ZERO for git and version control components of assignments and final test.

- Using Git and GitHub with Visual Studio 2026 for Windows:
 - <https://learn.microsoft.com/en-au/visualstudio/version-control/git-with-visual-studio?view=visualstudio>
- Using Git and GitHub with Visual Studio Code:
 - <https://code.visualstudio.com/docs/sourcecontrol/overview>

Exercise 7 Namespaces

You have already seen the use of the namespace in the previous exercises. Try to figure out the following:

1. Why would one use *nested namespaces*? Explain your answer.
2. Write a C# code snippet to explain how the nested namespaces can be used?

Exercise 8 How much of OO do you remember? (**Self-Exercise**)

The next tutorial / lab sheet will be a lot more intense and C# focused. C# is a fully object-oriented language.

1. How much of OO do you remember from Java days?
2. List some of the bad programming practices which makes the code less OO.

It is time to look at your OO skills. In your free time, revise what you were taught in Java courses.